التصحيح: الحلقات، التسليمات وقائمة الأشكال

الدرس 14 — 1IOT، SST-ICT-01 — أسئلة عن الواجبات، قواعد التسليم، التمرين 6 من الواجب C01

26/10/2026

1 ماذا فعلنا اليوم

لم نبدأ اليوم موضوعًا جديدًا: كان درس **تصحيح**. في البداية كتبت على السبورة **قائمة الطلبات** الآتية من الواجبات المنزلية ومن أسئلتكم:

- **الإزاحة** (القسم 2)؛
- **while** (القسم 3)؛
- **النقاط** بدلًا من الفراغ (القسم 4)؛
- **do while** (القسم 5)؛
- **تقرير عن الأخطاء** في التمارين (القسمان 6 و 7).

ثم أجريتم في القسم **فحص** الواجب المنزلي، وفي الجزء الثاني **صححنا معًا التمرين 6** من الواجب C01، **قائمة الأشكال** (القسم 8)، وهو الأصعب في السلسلة. أما الدالة ومعاملاتها، المقررة لليوم، فسنراها في الدروس القادمة.

2 الإزاحة

الإزاحة (**indentatura** أو **indentazione**) هي طريقة كتابة الأوامر **بدفعها إلى الداخل**، أي ليست كلها محاذية للحافة اليسرى: مثلًا، بعد `if (x == 3)` والقوس المفتوح، لا أكتب `printf` تحت القوس، بل **مدفوعًا إلى اليمين**، تاركًا فراغًا بين الحافة وبداية النص.

2.1 في C لا تتغير النتيجة

في C **لا معنى تنفيذيًا** للإزاحة: يمكنني كتابة البرنامج كله على **سطر واحد** وسيعمل. وليس الأمر كذلك في لغات أخرى: في **Python** (لغة برمجة أخرى، تُدرس مثلًا في بعض الثانويات) لا توجد أقواس معقوفة، وتفيد الإزاحة الآلة في فهم أن أمرًا **يعتمد** على تعليمة: فكل الأوامر المزاحة تحت `if` تُجمع داخل تلك `if`.

في C تقوم **الأقواس المعقوفة** بهذا العمل. لننظر إلى مثال السبورة:

```c
int main()
{
    int x;
    x = 3;
    if (x == 3)
    {
        printf("Inserisci un numero: ");
        scanf("%d", &n);
        printf("...");
    }
    printf("...");
}
```

الأوامر الثلاثة داخل القوسين (printf، scanf، printf) **تعتمد** **ثلاثتها** على أن x يساوي 3: إذا كان x يساوي 3 ينفذها كلها، وإذا كان 7 **يتخطاها**. أما printf بعد القوس المغلق فيُنفذ دائمًا. تفيد **الحاوية** المكونة من القوسين في تحديد جزء من الكود ومعاملته **كأنه** **تعليمة** **واحدة**.

تعريف

الإزاحة تدفع إلى اليمين بمقدار 3–4 رموز التعليمات الموجودة **داخل** كتلة أقواس، لتظهر فورًا أن هذه المجموعة من التعليمات **مجموعة** **قائمة** **بذاتها**. في C تفيد **من** **يقرأ** الكود فقط، لا الآلة.

ملاحظة

لماذا يبدأ السطر بعد if مزاحًا أصلًا؟ لأن if تكون عادةً **داخل** قوسي main: فأمامها إزاحة، وما بداخلها يأخذ **إزاحة** **أخرى**. كل مستوى من الأقواس يضيف إزاحة.

2.2 المفتاح Tab

عادة نستخدم للإزاحة **الجدولة**، أي المفتاح **Tab** (ذو السهمين، فوق مفتاح الأحرف الكبيرة، على اليسار). يدفع النص تلقائيًا بعض الفراغات ويعامله **كرمز** **واحد**: إذا قمت بجدولة ثم عدت إلى الوراء، يعود المؤشر ضربة كاملة، ولا يتوقف في المنتصف.

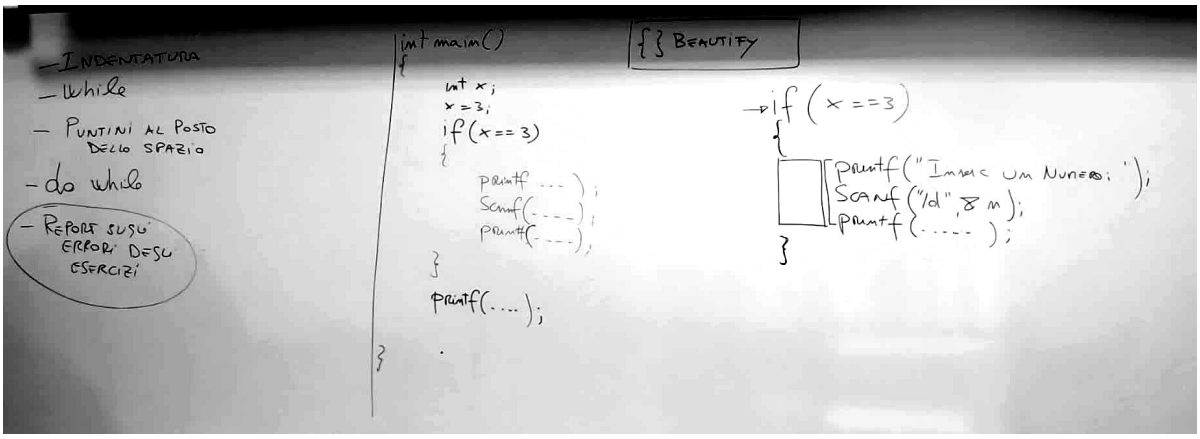
عرض الجدولة يعتمد على **المحرر**: مفكرة Windows تعرضها بعرض 8 رموز (قلت في القسم 5–6)، ومحررات أخرى 2 أو 3 أو 4. وعلى onlinegdb يفترض أن تكون 4. لا يغير ذلك شيئًا عندنا: المهم استخدامها.

2.3 الزر Beautify

عند الانتقال إلى سطر جديد داخل كتلة، يضع onlinegdb المؤشر من تلقاء نفسه **في** **الموضع** **الصحيح**. إضافة إلى ذلك، في onlinegdb (وبرامج أخرى سنستخدمها) زر ينفذ الإزاحة الصحيحة **تلقائيًا**: اسمه **Beautify** وهو في الأعلى، برمز قوسين معقوفين {}.

تنبيه

لا **يعرف** Beautify ما كان في رؤوسكم: فهو يزيح بحسب القواعد، بناءً على **ما** **هو** **مكتوب**. إذا أخطأتم في قوس فسيزيح بشكل خاطئ أيضًا، وليس الذنب ذنبه. ولهذا بالذات هو مفيد: إذا انتهى بعد Beautify printf كان يجب أن يكون خارج if **مدفوعًا** **إلى** **الداخل**، أو بقي القوس الأخير وحده، فقد وجدتم **الخطأ**.



شكل 1: سبورة بداية الدرس: قائمة الطلبات، ومثال `if` المزاحة داخل `main`، والزر Beautify. على اليمين كتلة التعليمات الثلاث التي تعتمد على `if`.

2.4 الحيلة: البنية أولًا

حتى لا تنسوا الأقواس توجد حيلة تصلح دائمًا: قبل كتابة الكود **اكتبوا البنية**.

1. أكتب `if`، قوسًا عاديًا مفتوحًا، قوسًا عاديًا مغلقًا.
2. أنتقل إلى سطر جديد، **قوسًا معقوفًا مفتوحًا**، **قوسًا معقوفًا مغلقًا**.
3. بعد ذلك فقط أدخل بين الأقواس و**أملأ الفراغات**: الشرط، `printf`، `scanf`.

أما إذا كتبت `if (x == 3)`، وفتحت القوس ثم `printf` و`scanf` وغيرهما، فمن السهل **نسيان قوس** الإغلاق في الطريق.

2.5 لماذا نفعل ذلك إذا كان البرنامج يعمل على أي حال؟

قد يقول أحدهم: إذا كان البرنامج يعمل، فلماذا أهتم بالإزاحة؟ أنا أفهمه، فليتدبر الآخرون أمرهم. الأمر لا يسير هكذا:

- عمل المبرمج **ليس عملًا فرديًا**: يُعمل ضمن **فريق**، ويقرأ الكود ويعدله آخرون أيضًا؛
- وحتى من يعمل وحده سيضطر عاجلًا أو آجلًا إلى **العودة** إلى كود كتبه **قبل ثلاث سنوات**: ولا يمكن تذكر كل شيء.

لذلك فالإزاحة **أساسية**، حتى لو لم تغير النتيجة.

مثال

سؤال اختبار. هل تؤثر الإزاحة في نتيجة البرنامج؟
الجواب: لا. تؤثر فقط في **قابلية القراءة**، أي التنظيم البصري للكود.

3 while: ماذا تفعل فعلًا

على السؤال «ماذا تفعل while؟» جاءت في القسم إجابات مثل «تكرر حتى يصبح صحيحًا» أو «تكرر السؤال حتى تُدخل المعلومة الصحيحة». هذه إجابات مرتبطة **بالمثال** الذي كنا نعمل عليه (التحقق من المعلومة)، وليس **بما**

هي while.

تعريف

while هي **بنية**، أي تركيب في البرمجة، تسمح **بتكرار** جزء من الكود **ما دام الشرط صحيحًا**. ليس «ما دام يوجد شرط»: بل ما دام الشرط **صحيحًا**. وعندما يصبح **خطأ** تتوقف.

3.1 مثال الخزانة

أبرمج آلة، مساعدًا، وأقول له: «**امشِ** حتى تصل إلى الخزانة». يخطو خطوة ويسأل نفسه: هل وصلت إلى الخزانة؟ لا. فيكرر. خطوة أخرى: هل وصلت؟ لا. وهكذا. كم خطوة سيخطو؟ لا نعرف: لكنه **عند كل خطوة يفحص** هل وصل. وعندما يتغير الجواب **يتوقف** عن المشي.

تنبيه

للدقة، السؤال الصحيح هو **بالعكس**: «**ألم** أصل بعد إلى الخزانة؟». ما دام الجواب **صحيحًا** (نعم، صحيح أنني لم أصل)، أخطو خطوة أخرى. وعندما أصل تصبح «لم أصل» **خطأ**، فأتوقف. في while **الشرط الصحيح** هو الذي يجعلها تكرر دائمًا.

3.2 while و for هما الشيء نفسه

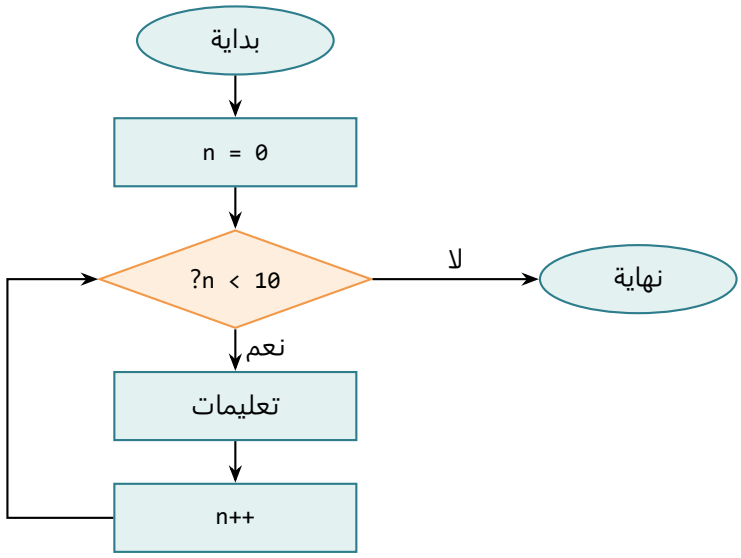
```for
for (int n = 0; n < 10; n++)
{
    // ... تعليمات ...
}
```

```while
int n = 0;
while (n < 10)
{
    // ... تعليمات ...
    n++;
}
```

شكل 2: الحلقة نفسها مكتوبة بـwhile وبـfor، كما على السبورة (while للحلقة الأولى و for للثانية).

في **while**، بين القوسين، يوضع **شرط التكرار**: `n < 10`، `n != 7`، أي شرط، بحسب ما نفعله. وداخل الكتلة يوضع **المعدِّل**: `n++`، `n = n * 2`، `n = n + 2`، `n = n * 3 - 4 + 18 + k`، ما تشاؤون. المهم أن **يتغير n**، وإلا يبقى الشرط صحيحًا إلى الأبد.

for تفعل الشيء نفسه: ما دام الشرط الموضوع **في الوسط** صحيحًا تكرر. الفرق في الكتابة فقط: في for توجد **التهيئة** و**الشرط** و**المعدِّل** كلها بين القوسين.



شكل 3: المخطط الانسيابي المشترك للحلقتين: يُفحص الشرط **قبل** كل تكرار.

3.3 متى for ومتى while

إنها مسألة **أسلوب**. عادةً:

- نستخدم **for** عندما نعرف **مسبقًا** كم مرة نكرر: تدريب فيه كل مرة **عشر** **قفزات**؛ أعرف أنها عشر؛
- نستخدم **while** عندما **لا** **نعرف** كم مرة نكرر: القفز **حتى** **الإنهاك** يعتمد على مقدار التحمل.

إنه تفضيل فقط: لا شيء يمنع استخدام while مع عدد ثابت أو for دون معرفة عدد المرات، سوى **القواعد** **غير** **المكتوبة** في البرمجة، التي يتبعها الجميع، مثل قواعد حسن السلوك.

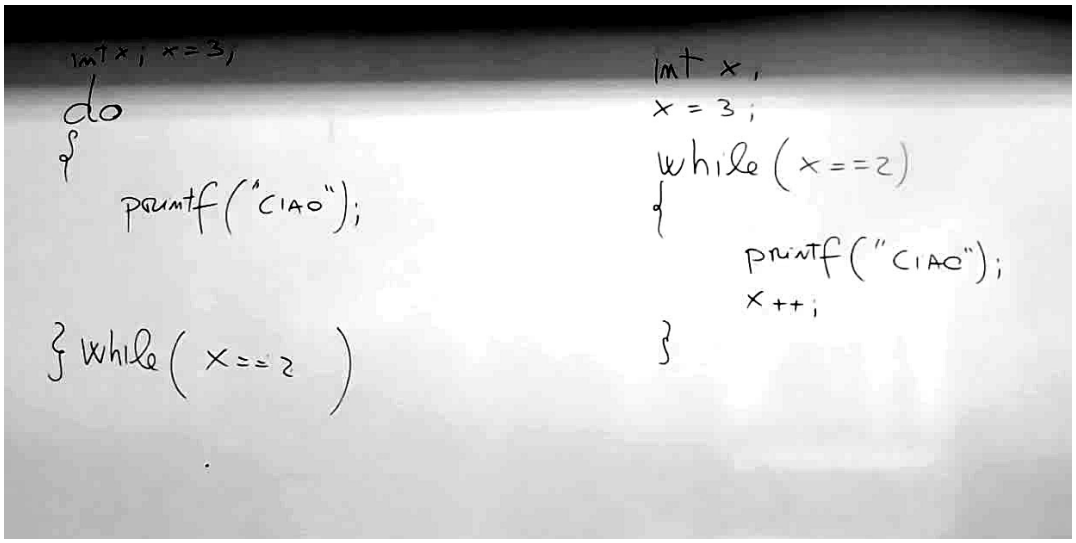
4 النقاط بدلًا من الفراغ

في تمارين الأشكال تُطبع في مواضع نجمة، `printf("*");` وفي مواضع أخرى **فراغ**، `printf(" ");`. وفي نص التمارين كانت توجد **نقاط** بدلًا من الفراغات. والسبب كان مكتوبًا في النص: الفراغ **لا** **يُرى**، ويصعب فهم هل يوجد فراغ في نقطة ما أم لا. تفيد النقطة فقط في **شغل** **مكان** **الفراغ** لجعله مرئيًا.

في البرنامج، حيث توجد نقطة في الرسم، يُطبع **الفراغ**. لكن إذا طبعتم النقاط فلا مشكلة: هدف التمرين هو **التدرب** **على** **حلقات** التكرار، وعلى `if`، وعلى **التداخل**. وبدلًا من النجمة يمكنكم وضع # أو @ أو ما تشاؤون: المهم أن تكون الحلقات صحيحة.

5 do while و while: الفرق الوحيد

تكرر **do** **while** الكتلة بين القوسين ما دام الشرط، المكتوب **في** **النهاية**، صحيحًا. وداخل الكتلة يمكن أن يوجد **أي** **شيء**: في التمارين كان يوجد غالبًا `printf` و`scanf`، لكن هذا لا يعني أنه سيكون دائمًا كذلك. إلى هنا هي مثل while. يوجد **فرق** **واحد** **فقط**: قد **لا** **تفعل** while **شيئًا** من أول مرة، أما do while فلا.



شكل 4: مثال السبورة: على اليسار do while، وعلى اليمين while، بالشرط نفسه x == 2 وقيمة x هي 3.

```c
int x;
x = 3;
while (x == 2)
{
    printf("ciao");
    x++;
}
```

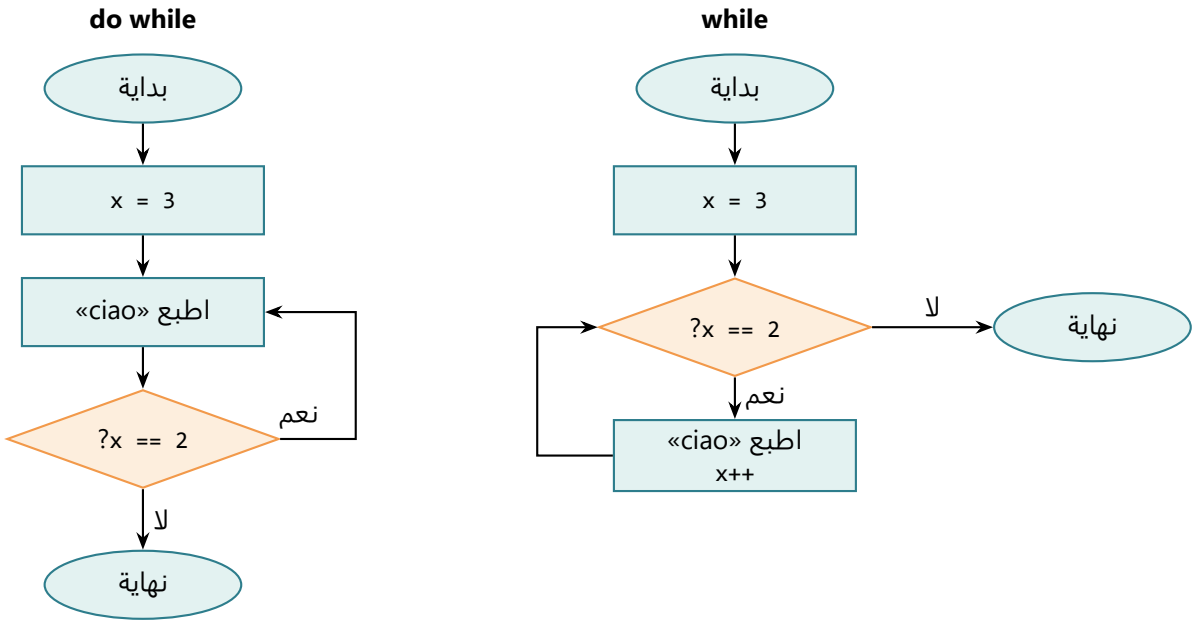
```c
int x;
x = 3;
do
{
    printf("ciao");
} while (x == 2);
```

مثال

كم مرة يطبع «ciao»؟

- **while** (الكود الذي فيه `while` في البداية): x يساوي 3 وتقول while «كرر ما دام x يساوي 2». الشرط **خاطئ أصلًا** في البداية: يقفز فورًا إلى النهاية و**لا يطبع شيئًا**. **صفر مرة.**
- **do while** (الكود الذي يبدأ بـdo): الشرط نفسه، خاطئ دائمًا، لكن للوصول إلى فحصه يجب أن **يمر** البرنامج **بالكتلة**: فيطبع «ciao» واحدة. **مرة واحدة.**

do while كأننا نكتب `printf("ciao")` **مرة واحدة** ثم ننفذ while.



شكل 5: في do while تأتي الطباعة **قبل** السؤال، وفي while **بعده**: مع x = 3 تطبع الأولى مرة واحدة، والثانية لا تطبع أبدًا.

ملاحظة

متى نستخدم do while؟ إذا كان يجب تنفيذ الكتلة **مرة واحدة على الأقل** فالأفضل **do while** (مثل سؤال للمستخدم، يجب طرحه مرة على الأقل). يمكن الحصول على النتيجة نفسها بـwhile، لكن يجب تحضير المتغيرات بطريقة «ذكية» قبل الحلقة. وإذا لم نعرف هل يجب تنفيذ الكتلة نستخدم **while**.

ملاحظة

Polling. أسأل أحيانًا في القسم «**polling؟**»: إنه اسم **طريقة** تقوم على **السؤال على فترات** هل يوجد جديد؛ وإذا لم يجب أحد فلا جديد. هذا ما تفعلونه بالهاتف عندما تنظرون إليه كل دقيقة لتروا هل وصلت رسالة. وفي القسم تعني: «هل أنتم معي؟ هل توجد أسئلة؟»

6 تسليم الواجبات المنزلية

تُقيَّم الواجبات المنزلية **فعلًا**، ويفيد الفحص في القسم في معرفة هل **أنجزتموها بأنفسكم**. إذا تبين من الفحص أن واجبًا مسلمًا صحيحًا بالكامل لم ينجزه من سلمه، **ينخفض** التقييم كثيرًا. تصحيح أخطائك جيد: **التصحيح** يعني إصلاح ما أخطأت فيه. أما تسليم عمل لا تعرف شرحه فأمر آخر.

6.1 الموعد النهائي موعد نهائي

للتسليم **وقت محدد**: إذا كان الموعد النهائي الساعة 20:00، فآخر لحظة ممكنة هي **19:59:59**. وفي 20:02 يكون **متأخرًا**.

مثال

المناقصة العامة. بين حين وآخر تنشر الدولة أو الإقليم أو البلدية **مناقصة** (**bando**): طلبًا عامًا يقولون فيه «نبحث عمن ينجز لنا هذا العمل، أرسلوا لنا عرض سعر»؛ ومن يقدم أفضل عرض يأخذ العمل. إذا كنت تعمل في شركة وكانت المناقصة مهمة، فهي طريقة للربح. لكن إذا وصل الطلب **ولو** **بعد** **دقيقة** من الموعد النهائي **فلا** **يُنظر** **فيه** **أصلًا**: ويضيع العمل. يجب **احترام** مواعيد التسليم دائمًا.

6.2 الواجب القادم

تصل المجموعة الجديدة من التمارين (3–5 تمارين) على Classroom وتُسلَّم **قبل** **نهاية** **السبت**، ليبقى يوم الأحد للتصحيح. إذا كانت لديكم مشكلة (لا حاسوب، لا اتصال)، **فهذا** **ليس** **عذرًا**: سلّموا قبل الموعد النهائي، أو استعيروا حاسوبًا من أحد، في بيت صديق، وأرسلوا العمل بالبريد قبل مساء السبت. لا يهم كيف: المهم أن **يصل**.

تنبيه

الواجبات المنزلية **إلزامية**. ومن يقول العكس يقول شيئًا **خاطئًا**.

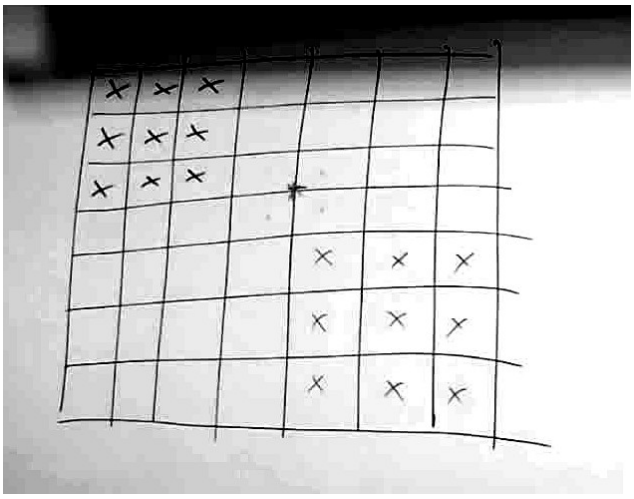
7 رسم المخرجات على شبكة

في الفحص في القسم كان لكل واحد تمرين **مختلف** (مستطيل، مربع، مثلث...) وسؤال مثل هذا:

مثال

«إذا كتب المستخدم 3 كضلع، ماذا يطبع البرنامج؟ ارسمه هنا: **رمز** **واحد** **في** **كل** **مربع**، **اترك** **الفراغات** **خالية**.»

كانت أشكال كثير من الرسوم صحيحة، لكن في **المكان** **الخطأ**: مثلًا مستطيل 3×3 مرسوم في **وسط** الشبكة، قبله أربعة مربعات فارغة.



شكل 6: على السبورة: 3×3 الصحيح، في الأعلى يسارًا، و3×3 نفسه منقولًا إلى وسط الشبكة، وهو ما لا يطبعه البرنامج أبدًا.

الحاسوب **لا** **ينقل** الرسم لأن «الزاوية قبيحة»: بل يطبع **من** **بداية** **السطر**، أي **في** **الأعلى** **يسارًا**، ولا يتقدم إلا

بالفراغات والأسطر الجديدة التي طلبنا طباعتها. يجب أن نتعلم التفكير مثل الحاسوب، الذي لا يفسر: بل ينفذ ما نقوله له. (الكود يُكتب ويُطبع دائمًا من اليسار إلى اليمين.)

قواعد رسم المخرجات:

1. نبدأ من المربع في الأعلى يسارًا؛
2. رمز واحد في كل مربع، مكتوب داخل المربع وليس على تقاطع الخطوط، وإلا لا يُفهم في أي من المربعات الأربعة المحيطة أردتم وضعه؛
3. حيث يطبع البرنامج فراغًا يبقى المربع خاليًا: بلا نقاط، إذا قال المطلوب «اترك الفراغات خالية».

ملاحظة

الشطيرة. أطلب شطيرة بالباذنجان فيعطونني شطيرة بشيء آخر: لماذا؟ إذا لم يتوفر الباذنجان يُسأل الزبون، ولا يُغير الطلب من تلقاء النفس. وكذلك التسليمات: نفعل ما هو مكتوب، وإذا وُجد شك نسأل. هذه الأخطاء ليست عقابًا: بل تفيد في تعلم الانتباه إلى التفاصيل، لأن الشكل الصحيح في المكان الخطأ يضر بالتقييم على أي حال.

8 التمرين 6 من C01: قائمة الأشكال

عندما يتوفر الوقت يكون تصحيح الواجبات في القسم، كما اليوم؛ وإلا يصلكم فقط ملف التصحيحات. في الجزء الثاني من الدرس صححنا معًا على onlinegdb التمرين 6 من الواجب C01 (في مستند الواجب هو الفصل 7)، وهو الأطول في السلسلة. النص، منسوخًا كتعليق في بداية البرنامج:

مثال

النص. اكتب برنامجًا فيه قائمة تتكرر حتى يختار المستخدم 0: 1 = مربع، 2 = مربع فارغ، 3 = مثلث، 0 = خروج.

- مع 1 أو 2 أو 3 يطلب الضلع (من 1 إلى 9، مع التحقق) ويرسم الشكل: مربع ممتلئ، مربع فارغ (الحافة فقط)، مثلث قائم؛
- مع عدد غير موجود في القائمة يطبع **Scelta non valida**؛
- يعدّ البرنامج الأشكال التي رسمها؛ وعند اختيار 0 يطبع `Figure disegnate:` والعدد.

تلميح: switch داخل do while، مثل قائمة الدرس 12. يبدأ العدّاد من 0 قبل الحلقة ويزيد 1 (`figure++`) في كل مرة يُرسم فيها شكل. يمكنك كتابة الرسومات الثلاثة في ثلاث حالات case منفصلة، بنسخ البرامج التي كتبتها سابقًا.

8.1 حلقة القائمة: do while

يجب أن تتكرر القائمة، ويجب طرح السؤال مرة واحدة على الأقل: نحتاج إلى do while. لا نخرج إلا عندما يختار المستخدم 0، إذن نكرر ما دام الاختيار مختلفًا عن 0: `while (scelta != 0)`.

داخل الحلقة نطلب الاختيار بـ`printf`، ويُستحسن أن نكتب فيه أيضًا الخيارات، ليعرف المستخدم ما يمكنه اختياره (**Fai una scelta** = قم باختيار)، ونحصل عليه بـ`scanf`.

```c
int scelta;

do
{
    printf("Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): ");
    scanf("%d", &scelta);

    // ... هنا الأشكال ...

} while (scelta != 0);
```

8.2 الخيارات: switch

يقترح النص **switch**، «مثل قائمة الدرس 12». أمام اقتراح كهذا، أول ما يجب فعله هو **فتح الملزمة A12** والبحث عن مثال القائمة: هناك توجد البنية المطلوبة بالضبط. يمكن فعل ذلك بـ`if` أيضًا، لكن switch أكثر **أناقة للخيارات المتعددة**، لأنها توجزها.

يكون switch على المتغير الذي حفظنا فيه الاختيار، أي `scelta`. الحالات هي:

- `case 1:`، `case 2:`، `case 3:` — الأشكال الثلاثة؛
- `case 0:` — **الإغلاق** (يُطبع عدد الأشكال المرسومة)؛
- `default:` — **كل الأعداد الأخرى**: «Scelta non valida».

النقطتان بعد `case 1` جزء من كتابة البنية، مثل `break` في نهاية كل حالة.

8.3 الضلع: أين نطلبه؟

مع 1 و2 و3 يجب طلب الضلع (**lato**) و**التحقق منه** (من 1 إلى 9). هناك ثلاث إمكانيات:

1. كتابة `printf` و`scanf` والتحقق do while **ثلاث مرات**، داخل كل case: يجب تجنبها. إذا كتبت الشيء نفسه ثلاث مرات **أخاطر بالخطأ ثلاث مرات**، وإذا أردت تعديلًا فعلي أن أجريه **ثلاث مرات**؛
2. وضع `if` **قبل switch** يطلب الضلع فقط إذا كان الاختيار 1 أو 2 أو 3: وهذا هو الحل **المفضل**؛
3. وضع `case 1:` و`case 2:` و`case 3:` **بالتسلسل**، بلا `break`: يمكن ذلك، لكن للتمييز بين الأشكال نحتاج إلى `if` تحتها على أي حال، ويصبح الكود أقل نظافة.

في الحل الثاني: إذا كان الاختيار بين 1 و3 يُطلب الضلع؛ وإذا كان 0 (خروج) أو عددًا غير صالح فلا حاجة إلى الضلع.

```c
if (scelta >= 1 && scelta <= 3)
{
    do
    {
        printf("Inserisci il lato (da 1 a 9): ");
        scanf("%d", &lato);
    } while (lato < 1 || lato > 9);
}
```

تنبيه

في القسم، كمحاولة أولى، خطر لـdo while شرط الاختيار (`scelta >= 1 && scelta <= 3`): وهذا خطأ، لأننا استخدمناه في `if`. شرط do while يخص **الضلع**، لا الاختيار: نكرر السؤال **إذا** **كان** **الضلع** **خاطئًا**، أي `lato < 1 || lato > 9`. إذا طلبت عددًا من 1 إلى 9 وكتب المستخدم 18، يتكرر السؤال.

يمكن حل التمرين بطرق أخرى، وهذا ليس خطأ: لكن سلسلة التمارين بُنيت على **switch** و**for** و**while**، ويجب حل التمرين بها. إذا بدا الأمر معقدًا الآن: بعد ثلاثة أشهر، عندما تنظرون إلى الوراء، سيبدو سهلًا.

8.4 الحالة 1: المربع الممتلئ

لرسم **ببعدين** نحتاج إلى **for** **متداخلتين**: الخارجية على الأسطر (y)، والداخلية على الأعمدة (x). في المربع تصل الاثنتان إلى `lato`، لأن الأضلاع متساوية.

```c
case 1:
    for (int y = 0; y < lato; y++)
    {
        for (int x = 0; x < lato; x++)
        {
            printf("*");
        }
        printf("\n");
    }
    nFigure++;
    break;
```

تنتج for على x **سطرًا**؛ ويتكرر هذا السطر مع y تساوي 0، 1، 2، 3، 4 (مع الضلع 5): تتوقف عند 4 لأنها تبدأ من 0. `printf("*")` هو التعليمة الوحيدة في for الداخلية: لا تحتاج إلى أقواس، لكننا **نضعها** **دائمًا**. وبعد القوس المغلق لـ`for(...)` **لا** **توضع** فاصلة منقوطة: فوجود ; هناك يعني «كرر تعليمة فارغة»، وتُنفذ الكتلة بين الأقواس مرة واحدة فقط.

ملاحظة

التجربة قطعة قطعة. بعد كتابة `case 1` فقط، يُجرَّب البرنامج باختيار **1** **فقط**: لم يُعالج 2 و3 بعد، ومع اختيار آخر لا يفعل البرنامج شيئًا (أو يفعل أشياء غريبة). وكالعادة: إذا كُتب في `scanf` رمز غير صحيح، مثل **حرف** بدلًا من العدد، **يجن** **جنون** البرنامج.

تنبيه

في **نهاية** **كل** **سطر**، أي بعد for على x وداخل for على y، نحتاج إلى `printf("\n")` **للانتقال** **إلى** **سطر** **جديد**: وبدونه تنتهي كل النجوم **متتالية** على السطر نفسه.

8.5 الخطأ الأول: lato غير معرَّف

في أول تجربة **لم** **يُترجم** البرنامج. في القسم كنت قد عرّفت `int lato;` **داخل** **قوسي** **if**، قبل do while مباشرة:



شكل 7: onlinegdb: `int lato;` في السطر 29، داخل قوسي `if`. وفي السطر 40 يشير المترجم إلى `error: 'lato' undeclared`.

المتغير المعرَّف **داخل** **كتلة** **أقواس** يوجد **داخل** **تلك** **الكتلة** **فقط**: وعندما تنتهي الكتلة **يُدمر** المتغير. وswitch خارج `if`، فهناك **لا** **يوجد** `lato` بعد الآن، ويقول المترجم إنه غير معرَّف.

تعريف

التصحيح: انقل `int lato;` **إلى** **أعلى** `main`، مع `int scelta;`، خارج كل كتلة. وهكذا يكون المتغير مرئيًا في `main` كله.

8.6 عندما لا يُترجم البرنامج

في القسم لم تعمل بعض البرامج حتى بعد هذا التصحيح. ما يجب فعله، بالترتيب:

1. **اقرأ** **رسالة** الخطأ: مثلًا قال onlinegdb إن **قوسًا** **ناقص**.
2. اضغط **Beautify** وانظر إلى الإزاحة: في إحدى الحالات كان هناك قوس **زائد**، تحت `for`، يجب حذفه، ثم كان ينقص قوس إغلاق **في** **النهاية**، بعد آخر سطر. انتبه إلى القراءة الجيدة قبل التصحيح: في القسم قرأت خطأ في البداية، فظننت قوسًا صحيحًا خاطئًا، واضطررت إلى الإلغاء بـ`Ctrl+Z`. مع الإزاحة يُرى فورًا أي كتلة غير مغلقة.
3. افحص كتلة كتلة **أين** **يُغلق** كل قوس: قوس `main`، وقوس `do`، وقوس `switch`.
4. **قارن** كودك، سطرًا سطرًا، بالبرنامج الصحيح (القسم 8.10).

> **ملاحظة**
>
> **خطأ لا يُرى.** في إحدى الحالات أشار المترجم إلى خطأ في سطر يبدو صحيحًا تمامًا، ربما بسبب رمز غير معروف: بحذف ذلك الجزء من الكود يعمل البرنامج، وبإعادته لا يعمل. قد يحدث ذلك عندما يدخل في الكود رمز غريب أو غير مرئي (مثلًا عند نسخ نص من مكان آخر). النصيحة: احذف ذلك الجزء وأعد كتابته يدويًا.

8.7 الحالة 2: المربع الفارغ

المربع الفارغ يبقى مربعًا: تُنسخ for المتداخلتان من الحالة 1. الفرق أنه يُطبع أحيانًا نجمة وأحيانًا فراغ.

تُطبع النجمة على الحافة فقط، أي في أربع حالات (مع الضلع 5، تمتد x وy من 0 إلى 4):

- `x == 0`: العمود الأول؛
- `y == 0`: السطر الأول؛
- `x == lato - 1`: العمود الأخير (الضلع ناقص 1، لأننا لا نصل إلى 5 بل إلى 4)؛
- `y == lato - 1`: السطر الأخير.

في كل الحالات الأخرى يُطبع فراغ. يجب طباعة الفراغ، حتى لو لم يُرَ: وإلا تنزلق نجوم الجهة اليمنى إلى اليسار، كما حدث في أحد تمارين الدرس الماضي.

```
case 2:
    for (int y = 0; y < lato; y++)
    {
        for (int x = 0; x < lato; x++)
        {
            if ((x == 0) || (y == 0) || (x == lato - 1) || (y == lato - 1))
            {
                printf("*");
            }
            else
            {
                printf(" ");
            }
        }
        printf("\n");
    }
    nFigure++;
    break;
```

8.8 الحالة 3: المثلث

رسمنا المثلث في الدرس 13: إنه أيضًا for متداخلة مزدوجة، لكن على x نصل حتى y (`x <= y`): في السطر الأول نجمة، وفي الثاني نجمتان، وهكذا حتى النهاية.

```
case 3:
    for (int y = 0; y < lato; y++)
    {
        for (int x = 0; x <= y; x++)
        {
            printf("*");
        }
        printf("\n");
    }
    nFigure++;
    break;
```

8.9 default وعدّاد الأشكال

إذا لم يكن الاختيار 1 ولا 2 ولا 3 ولا 0، نصل إلى `default`، التي تطبع «Scelta non valida».

يجب أن **يعد** البرنامج أيضًا الأشكال المرسومة: نحتاج إلى متغير إضافي، `int nFigure = 0;`، يبدأ من **الصفر** **قبل** **الحلقة**. وفي كل حالة من حالات الأشكال الثلاث، **قبل** **break**، يزيد 1: `nFigure++;`. وعندما يختار المستخدم 0، تطبع `case 0` النتيجة (**Le figure disegnate sono** = عدد الأشكال المرسومة):

```
case 0:
    printf("Le figure disegnate sono %d\n", nFigure);
    break;
default:
    printf("Scelta non valida\n");
```

8.10 البرنامج الكامل

هذا هو البرنامج المكتوب في القسم، مع تصحيحات المربع التالي (مجرب بـgcc).

```c
#include <stdio.h>

int main()
{
    int scelta;
    int lato;
    int nFigure = 0;

    do
    {
        printf("Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): ");
        scanf("%d", &scelta);

        // الضلع مطلوب للأشكال الثلاثة فقط
        if (scelta >= 1 && scelta <= 3)
        {
            do
            {
                printf("Inserisci il lato (da 1 a 9): ");
                scanf("%d", &lato);
            } while (lato < 1 || lato > 9);
        }

        switch (scelta)
        {
            case 1:
                for (int y = 0; y < lato; y++)
                {
                    for (int x = 0; x < lato; x++)
                    {
                        printf("*");
                    }
                    printf("\n");
                }
                nFigure++;
                break;
            case 2:
                for (int y = 0; y < lato; y++)
                {
                    for (int x = 0; x < lato; x++)
                    {
                        if ((x == 0) || (y == 0) || (x == lato - 1) || (y == lato - 1))
                        {
                            printf("*");
                        }
                        else
                        {
                            printf(" ");
                        }
                    }
                    printf("\n");
                }
                nFigure++;
                break;
            case 3:
                for (int y = 0; y < lato; y++)
                {
                    for (int x = 0; x <= y; x++)
                    {
                        printf("*");
                    }
                    printf("\n");
                }
                nFigure++;
                break;
            case 0:
                printf("Le figure disegnate sono %d\n", nFigure);
                break;
            default:
                printf("Scelta non valida\n");
        }

    } while (scelta != 0);

    return 0;
}
```

تنبيه

خطآن بقيا في الكود المكتوب في القسم (لم يتسع الوقت لتجربته):

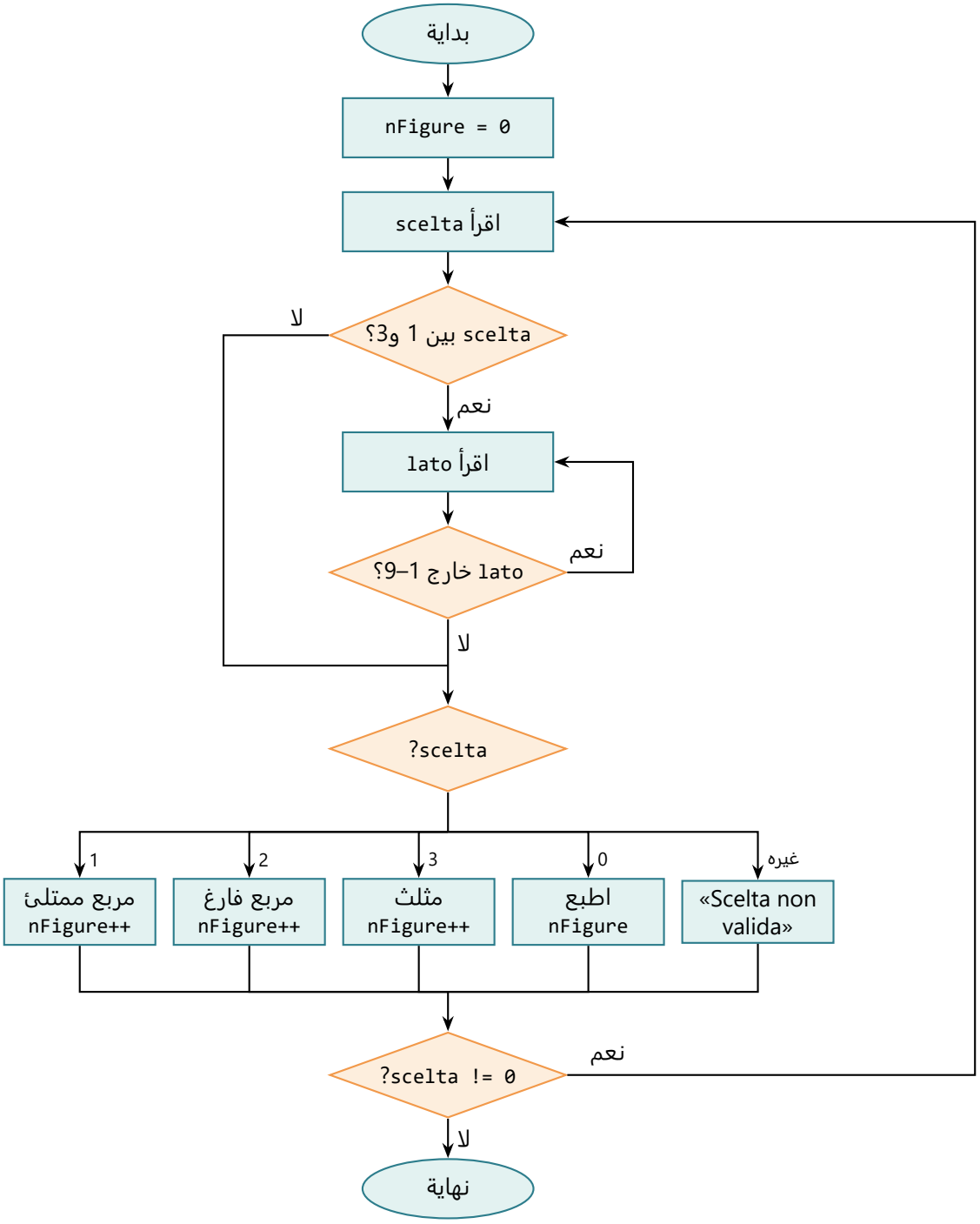
- في case 2 كان الشرط الأخير (y = lato - 1)، **بعلامة مساواة واحدة**. علامة = الواحدة **تسند** القيمة، **ولا تقارنها**: فيصبح الشرط صحيحًا دائمًا (مع ضلع أكبر من 1) ويتغير y أيضًا. مع الضلع 5 يطبع البرنامج **سطرين ممتلئين** من النجوم بدلًا من المربع الفارغ، و**لا يعطي** المترجم **أي خطأ**. للمقارنة نحتاج إلى **علامتي مساواة**: (y == lato - 1)؛
- في case 2 كانت **الفاصلة المنقوطة** ناقصة بعد nFigure++: وهذا يشير إليه المترجم.

إضافة إلى ذلك أضفت \n في نهاية رسالتي case 0 وdefault، لتبدأ القائمة التالية على سطر جديد.

8.11 تجربة

```
Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): 1
Inserisci il lato (da 1 a 9): 12
Inserisci il lato (da 1 a 9): 4
****
****
****
****
Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): 2
Inserisci il lato (da 1 a 9): 5
*****
*   *
*   *
*   *
*****
Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): 3
Inserisci il lato (da 1 a 9): 4
*
**
***
****
Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): 7
Scelta non valida
Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): 0
Le figure disegnate sono 3
```

8.12 المخطط الانسيابي



شكل 8: قائمة الأشكال: `switch` (المعيَّن «`scelta`؟») داخل do while، التي تتكرر ما دام الاختيار مختلفًا عن 0.

9 ما يجب تذكره من هذا الدرس

- في C لا تغير **الإزاحة** النتيجة، لكنها تجعل الكود **مقروءًا**؛ والزر **Beautify** ينظمها ويساعد في إيجاد الأقواس الخاطئة. تُكتب **البنية** (الأقواس) أولًا، ثم تُملأ.
- تكرر **while** كتلة **ما دام الشرط صحيحًا**؛ و**for** تفعل الشيء نفسه، مع التهيئة والشرط والمعدِّل بين القوسين. for عندما نعرف **كم مرة**، وwhile عندما لا نعرف.

- تنفذ **do while** الكتلة **مرة واحدة على الأقل**، حتى لو كان الشرط خاطئًا؛ وقد لا تنفذها while أبدًا.
- تُحترم **المواعيد النهائية** بالدقيقة؛ والواجبات المنزلية **إلزامية**.
- تُرسم المخرجات على شبكة ابتداءً من **الأعلى يسارًا**، **رمز واحد في كل مربع**، والفراغات **خالية**.
- المتغير المعرَّف **داخل كتلة** أقواس يوجد **هناك فقط**: وإذا لزم خارجها يُعرَّف في أعلى `main`.
- للمقارنة نحتاج إلى **علامتي مساواة** (==)؛ والعلامة الواحدة **تسند**، ولا ينتبه المترجم إلى ذلك.

واجب منزلي. تجدون على Classroom المجموعة الجديدة من التمارين: التسليم **قبل نهاية السبت**. قبل إنجازها، **قارنوا** التمرين 6 الخاص بكم بالبرنامج الكامل في هذه الملزمة وابحثوا عن موضع الخطأ.

10 مسرد: كلمات إيطالية تُستخدم في القسم

- **indentatura**: إزاحة
- **graffa**: قوس معقوف
- **consegna / scadenza**: تسليم / موعد نهائي
- **compito per casa**: واجب منزلي
- **griglia / quadretto**: شبكة / مربع صغير
- **quadrato pieno / vuoto**: مربع ممتلئ / فارغ
- **lato**: ضلع
- **Fai una scelta**: قم باختيار
- **esci**: خروج
- **contatore**: عدّاد
- **undeclared** (بالإنجليزية): غير معرَّف